AI Codebase Explainer — Ultimate Learning & Execution Roadmap

Goal: Build an AI-powered system that analyzes GitHub repositories and explains their structure, architecture, and functionality — while mastering backend engineering, data processing, static code analysis, AI integration, and full-stack development.

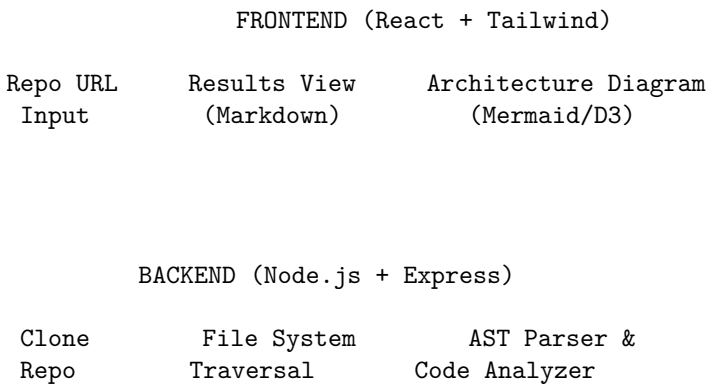
Total Duration: 25 Days (flexible — adjust to your pace) **Difficulty:** Intermediate | **Outcome:** Portfolio-ready, interview-worthy project

---

Master Timeline At a Glance

Week	Phase	Focus Area	Days
1	Phase 1 + 2	Backend Fundamentals + File System & Data Processing	1–6
2	Phase 3 + 4	Code Understanding (AST) + AI Integration	7–14
3	Phase 5 + 6	Frontend & Visualization + Final Polish	15–25

Architecture Overview



AI / LLM Engine
(OpenAI / Gemini)

PHASE 1: Backend Fundamentals

Days 1–3 | Difficulty: (Beginner-Friendly)

Learning Objectives

By the end of this phase, you should be able to: - Explain how Node.js handles asynchronous I/O using the event loop - Build REST APIs with Express.js from scratch - Understand middleware, routing, and the request-response cycle - Structure a backend project properly

---

Day 1: Node.js Core Concepts

What to Learn: - What Node.js is and why it exists (V8 engine, non-blocking I/O) - The Event Loop — how Node handles concurrent requests with a single thread - CommonJS modules (`require`/`module.exports`) vs ES Modules (`import`/`export`) - Core modules: `http`, `path`, `url`, `os`

Learning Resources:

Resource	Type	Link
Node.js Official Docs — Getting Started	Docs	https://nodejs.org/en/docs
"Node.js Tutorial for Beginners" — Programming with Mosh	Video (1hr)	Search "Mosh Node.js" on YouTube
"The Node.js Event Loop" — Node.js Docs	Deep Dive	https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick
"Node.js Event Loop Explained" — ByteByteGo	Visual	Search "ByteByteGo Node event loop" on YouTube

Practice Task:

```
1. Create a file called `server.js`
2. Use the built-in `http` module to create a server
3. Handle 3 different routes manually:
   - GET  /          → "Welcome to AI Codebase Explainer"
   - GET  /health    → { status: "ok", uptime: process.uptime() }
   - ANY  other      → 404 "Not Found"
4. Run it on port 3000
```

Self-Check: - [ ] Can you explain what happens when 1000 requests hit your server at the same time? - [ ] Can you draw the event loop phases from memory? (timers → pending → idle → poll → check → close) - [ ] Do you understand why `setTimeout(fn, 0)` doesn't execute immediately?

Day 2: Express.js & REST APIs

What to Learn: - Express.js setup and project structure - Routing (GET, POST, PUT, DELETE) - Middleware concept — how `next()` works - Request parsing: `req.params`, `req.query`, `req.body` - Response methods: `res.json()`, `res.status()`, `res.send()` - Error handling middleware

Learning Resources:

Resource	Type	Link
Express.js Official Guide	Docs	https://expressjs.com/en/starter/installing.html
"Express JS Full Course" — Dave Gray	Video (4hr)	Search "Dave Gray Express" on YouTube
REST API Design Best Practices "RESTful API Design" — Microsoft	Article Guide	https://restfulapi.net/ Search "Microsoft REST API guidelines"

Practice Task:

```
Build your project's initial API server with these endpoints:

POST   /api/analyze       → Accept { repoUrl } in body, return { message: "Analysis started'
GET    /api/analyze/:id   → Return { status: "pending", id }
DELETE /api/analyze/:id   → Return { message: "Analysis cancelled" }

Add these middlewares:
  - JSON body parser
  - Request logger (log method, url, timestamp)
  - Error handler (catch all errors, return 500)
```

Suggested Project Structure:

```
ai-codebase-explainer/
  src/
     server.js            # Entry point
     routes/
        analyze.js       # Analysis routes
     middleware/
        logger.js        # Request logging
        errorHandler.js  # Global error handler
     utils/
        helpers.js       # Utility functions
  package.json
  .gitignore
```

Self-Check: - [ ] What's the difference between `app.use()` and `app.get()`? - [ ] What does `next()` do and what happens if you forget to call it? - [ ] Can you explain the middleware execution order?

Day 3: Project Setup & Git Workflow

What to Learn: - Environment variables with `dotenv` - Nodemon for hot-reloading during development - Git basics: branching, committing, `.gitignore` - npm scripts and `package.json` configuration

Learning Resources:

Resource	Type	Link
dotenv npm package	Docs	https://www.npmjs.com/package/dotenv
Git & GitHub Crash Course — Traversy Media	Video (30m)	Search "Traversy Media Git" on YouTube
.gitignore templates	Reference	https://github.com/github/gitignore

Practice Task:

```
1. Initialize a Git repo for the project
2. Set up .env file with PORT=3000
3. Configure package.json scripts:
   - "start": "node src/server.js"
   - "dev": "nodemon src/server.js"
4. Create a proper .gitignore (node_modules, .env, etc.)
5. Test all 3 APIs with curl or Postman
6. Make your first meaningful commit
```

Self-Check: - [ ] Is your `.env` file in `.gitignore`? - [ ] Can you start the server with `npm run dev`? - [ ] Do all 3 API endpoints respond correctly?

Phase 1 — Common Mistakes to Avoid

> [!CAUTION] - **Don't skip understanding the event loop** — interviewers *love* asking about it - **Don't hardcode values** — use environment variables from day one - **Don't forget error handling** — always add try-catch in async routes - **Don't use `var`** — always use `const` and `let`

Phase 1 — Interview Connection

> **What interviewers ask:** "Explain how Node.js handles 10,000 concurrent connections with a single thread." **Your answer now:** You can explain the event loop, non-blocking I/O, and libuv thread pool.

PHASE 2: File System & Data Processing

Days 4–6 | Difficulty: (Intermediate)

Learning Objectives

- Clone GitHub repositories programmatically
- Traverse directory trees recursively
- Filter and categorize files intelligently
- Process large file systems efficiently

Day 4: GitHub Repo Cloning & fs Module Basics

What to Learn: - Node.js `fs` module — sync vs async vs promises API - `child_process` module — `exec` and `spawn` for running shell commands - Cloning repos with `git clone` via Node.js - Temporary directory management

Learning Resources:

Resource	Type	Link
Node.js File System Docs	Docs	https://nodejs.org/api/fs.html
Node.js `child_process` Docs	Docs	https://nodejs.org/api/child_process.html

Resource	Type	Link
“simple-git” npm package	Library	https://www.npmjs.com/package/simple-git
“Mastering the Node.js fs Module” — LogRocket	Article	Search “LogRocket Node fs module”

Practice Task:

```
Build a RepoCloner module:

1. Accept a GitHub URL (e.g., https://github.com/user/repo)
2. Validate the URL format (must be a valid GitHub URL)
3. Clone it into a temporary directory (use os.tmpdir())
4. Return the path to the cloned directory
5. Add cleanup function to delete cloned repos when done

// src/services/repoCloner.js
class RepoCloner {
  async clone(repoUrl) { /* ... */ }
  async cleanup(repoPath) { /* ... */ }
  validateUrl(url) { /* ... */ }
}
```

Self-Check: - [ ] What’s the difference between `fs.readFile` and `fs.readFileSync`? - [ ] Why should you prefer `fs.promises` over callback-based fs? - [ ] What happens if the clone fails? Is the error handled?

Day 5: Recursive Directory Traversal

What to Learn: - Recursion fundamentals — base case and recursive case - `fs.readdir` with `withFileTypes: true` - Building a file tree data structure - Ignoring directories (node_modules, .git, dist, build)

Learning Resources:

Resource	Type	Link
“Recursion” — freeCodeCamp	Tutorial	Search “freeCodeCamp recursion JavaScript”
`glob` npm package	Library	https://www.npmjs.com/package/glob
“Tree Command in Node.js” — Tutorial	Hands-on	Build your own `tree` command

Practice Task:

```
Build a FileTraverser module:

1. Recursively walk through a directory
2. Build a tree structure like:
   {
     name: "src",
     type: "directory",
     children: [
       { name: "server.js", type: "file", extension: ".js", size: 1024 },
       { name: "routes", type: "directory", children: [...] }
     ]
   }
3. Skip these directories: node_modules, .git, dist, build, __pycache__
4. Collect stats: total files, total directories, file type counts

Expected output:
    Repository Statistics:
     Total Files: 47
     Total Directories: 12
     File Types: { ".js": 23, ".ts": 10, ".json": 5, ".md": 4, ".css": 5 }
     Ignored: 3 directories skipped
```

Self-Check: - [ ] Can your traverser handle deeply nested directories (10+ levels)? - [ ] What happens with symlinks? Have you handled circular references? - [ ] Is your solution efficient for repositories with 10,000+ files?

Day 6: File Filtering & Content Reading

What to Learn: - File extension filtering strategies - Reading file content safely (handle binary files, encoding) - File metadata extraction (size, last modified, line count) - Batching file reads for performance

Learning Resources:

Resource	Type	Link
`path` module in Node.js	Docs	https://nodejs.org/api/path.html
Stream Processing in Node.js	Advanced	https://nodejs.org/api/stream.html
"Data Processing Patterns" — Design Patterns	Concept	Search "data pipeline patterns"

Practice Task:

```
Build a FileFilter and ContentReader module:
```

```
1. FileFilter: Accept a config of file extensions to include/exclude
   - Include: .js, .ts, .py, .java, .go, .rb, .rs, .cpp, .c, .jsx, .tsx
   - Exclude: .min.js, .map, .lock, .log
   - Max file size: 100KB (skip larger files)

2. ContentReader: Read filtered files and extract:
   - File content (as string)
   - Line count
   - Import/require statements (simple regex)
   - Whether file has default export

3. Wire it into your API:
   POST /api/analyze → clone repo → traverse → filter → read → return summary
```

Self-Check: - [ ] What encoding issues might arise reading files? How do you handle them? - [ ] Can you process 500 files without running out of memory? - [ ] What's the time complexity of your traversal + filtering pipeline?

Phase 2 — Common Mistakes to Avoid

> [!WARNING] - **Don't read all files into memory at once** — use streams for large files - **Don't forget to clean up cloned repos** — you'll fill up disk space fast - **Don't ignore binary files** — checking for them prevents garbled output - **Don't hardcode file extensions** — make them configurable

Phase 2 — Interview Connection

> **What interviewers ask:** "How would you process a directory with 1 million files efficiently?" **Your answer now:** You can discuss recursive traversal, streaming, batching, memory management, and ignoring irrelevant directories.

PHASE 3: Code Understanding (AST Parsing)

Days 7–10 | Difficulty: (Advanced)

Learning Objectives

- Understand what an Abstract Syntax Tree (AST) is and why it matters
- Parse JavaScript/TypeScript code into AST nodes
- Extract functions, classes, imports, and exports programmatically

- Build a code structure map for any repository

Day 7: AST Fundamentals

What to Learn: - What is an AST? (Every compiler/linter/formatter uses them) - AST node types: Program, FunctionDeclaration, ClassDeclaration, ImportDeclaration - How tools like ESLint, Prettier, and Babel work under the hood - Difference between parsing and analysis

Learning Resources:

Resource	Type	Link
AST Explorer (interactive!)	Tool	https://astexplorer.net/
"ASTs & How to Use Them" — Superhero.js	Video	Search "Superhero.js AST" on YouTube
Babel Parser Docs	Docs	https://babeljs.io/docs/babel-parser
"Leveling Up Your JavaScript Skills with ASTs"	Article	Search "codeburst AST JavaScript"
Tree-sitter (multi-language)	Library	https://tree-sitter.github.io/tree-sitter/

Practice Task:

```
Explore ASTs hands-on:

1. Go to https://astexplorer.net/
2. Paste this code and study the AST output:

   import express from 'express';

   class Server {
     constructor(port) {
       this.port = port;
     }

     start() {
       const app = express();
       app.listen(this.port);
     }
   }
```

```
   export default Server;

3. Identify these nodes in the AST:
   - ImportDeclaration → what is imported and from where
   - ClassDeclaration → class name, methods, constructor
   - ExportDefaultDeclaration → what is exported

4. Write down the node types for each line of code
```

Self-Check: - [ ] Can you explain what an AST is to a non-programmer? - [ ] What's the difference between a `FunctionDeclaration` and `FunctionExpression` in the AST? - [ ] Why do tools like ESLint parse code into ASTs instead of using regex?

Day 8: Building the Code Parser

What to Learn: - Using `@babel/parser` to parse JavaScript/TypeScript - AST traversal with `@babel/traverse` - Extracting specific node types - Handling parse errors gracefully

Learning Resources:

Resource	Type	Link
@babel/parser npm	Docs	https://www.npmjs.com/package/@babel/parser
@babel/traverse npm	Docs	https://www.npmjs.com/package/@babel/traverse
Babel Plugin Handbook	Deep Dive	https://github.com/jamiebuilds/babel-handbook
"Write a Babel Plugin" — Kent C. Dodds	Video	Search "Kent C Dodds Babel plugin"

Practice Task:

```
Build a CodeParser module:

// src/services/codeParser.js

class CodeParser {
  parse(code, filename) {
    // Use @babel/parser to parse the code
    // Handle: .js, .jsx, .ts, .tsx files
    // Return the AST
  }
```

```
  extractFunctions(ast) {
    // Return array of:
    // { name, params, lineStart, lineEnd, isAsync, isExported }
  }

  extractClasses(ast) {
    // Return array of:
    // { name, methods: [...], properties: [...], isExported }
  }

  extractImports(ast) {
    // Return array of:
    // { source, specifiers: [...], isDefault, isNamespace }
  }

  extractExports(ast) {
    // Return array of:
    // { name, type: "default"|"named", isReExport }
  }
}

Test it on a real file from a popular open-source repo!
```

Self-Check: - [ ] Does your parser handle TypeScript syntax (type annotations, interfaces)? - [ ] What happens when the parser encounters invalid syntax? - [ ] Can you parse JSX?

Day 9: Dependency Graph Construction

What to Learn: - What a dependency graph is and why it matters - Building a module dependency map from imports/exports - Detecting circular dependencies - Identifying entry points and core modules

Learning Resources:

Resource	Type	Link
"Dependency Graph" — Wikipedia	Concept	https://en.wikipedia.org/wiki/Dependency_graph
madge — npm (visual dependency graphs)	Tool	https://www.npmjs.com/package/madge
Graph Data Structures — freeCodeCamp	Tutorial	Search "freeCodeCamp graph data structures"

Practice Task:

```
Build a DependencyAnalyzer module:

1. Take all parsed imports from every file
2. Resolve relative paths to absolute paths
3. Build a dependency graph:
   {
     "src/server.js": {
       imports: ["src/routes/analyze.js", "src/middleware/logger.js"],
       importedBy: [],
       isEntryPoint: true
     },
     "src/routes/analyze.js": {
       imports: ["src/services/repoCloner.js"],
       importedBy: ["src/server.js"],
       isEntryPoint: false
     }
   }
4. Identify: entry points, leaf modules, most-imported modules
5. Detect circular dependencies (A → B → C → A)
```

Self-Check: - [ ] Can you explain what a DAG (Directed Acyclic Graph) is? - [ ] How does your system handle `dynamic imports` (`import()`)? - [ ] What algorithm would you use to detect cycles? (Hint: DFS with visited tracking)

Day 10: Multi-Language Support & Structure Summary

What to Learn: - Tree-sitter for parsing multiple languages - Creating a unified code structure format regardless of language - Generating a human-readable code summary

Learning Resources:

Resource	Type	Link
Tree-sitter Introduction	Docs	https://tree-sitter.github.io/tree-sitter/
node-tree-sitter npm	Library	https://www.npmjs.com/package/tree-sitter
“Tree-sitter — A new parsing system”	Talk	Search “Tree-sitter Strange Loop” on YouTube

Practice Task:

```
Extend your system to produce a unified CodeStructure output:

{
  repository: "user/repo",
  language: "JavaScript",
  stats: { files: 47, functions: 123, classes: 15, imports: 89 },
  entryPoints: ["src/index.js"],
  modules: [
    {
      file: "src/server.js",
      functions: [...],
      classes: [...],
      imports: [...],
      exports: [...],
      complexity: "medium"  // based on function count, nesting depth
    }
  ],
  dependencyGraph: { ... },
  summary: "This is an Express.js web server with 5 routes and 3 middleware..."
}
```

Self-Check: - [ ] Does your unified format work for both JavaScript and Python files? - [ ] Can you generate a meaningful summary from just the structure (without AI)? - [ ] How would you estimate code complexity from the AST?

Phase 3 — Common Mistakes to Avoid

[!CAUTION] - **Don't try to parse every language from scratch** — use Tree-sitter for multi-language support - **Don't ignore arrow functions** — they're the most common function type in modern JS - **Don't forget destructured imports** — `import { a, b } from 'module'` - **Don't try to handle every edge case** — start with the 80% case, iterate later

Phase 3 — Interview Connection

What interviewers ask: "How do tools like ESLint or Webpack analyze code?" **Your answer now:** You can explain AST parsing, tree traversal, dependency resolution, and how every major dev tool is built on these concepts.

PHASE 4: AI Integration

Days 11–14 | Difficulty: (Advanced)

Learning Objectives

- Understand how Large Language Models (LLMs) work at a high level
- Master prompt engineering for code explanation tasks
- Design effective context windows for code analysis
- Handle rate limits, token budgets, and API costs

Day 11: LLM Fundamentals & API Setup

What to Learn: - How LLMs work: tokens, context windows, temperature, top-p - OpenAI API / Google Gemini API setup - Token counting and cost estimation - Rate limiting and retry strategies

Learning Resources:

Resource	Type	Link
OpenAI API Documentation	Docs	https://platform.openai.com/docs
Google Gemini API Documentation	Docs	https://ai.google.dev/docs
"Prompt Engineering Guide" — DAIR.AI	Guide	https://www.promptingguide.ai/
OpenAI Tokenizer Tool	Tool	https://platform.openai.com/tokenizer
"Building AI Apps" — Fireship	Video (12m)	Search "Fireship OpenAI API" on YouTube

Practice Task:

```
1. Sign up for OpenAI API or Google Gemini API (free tiers available)
2. Create an API wrapper module:

// src/services/aiEngine.js

class AIEngine {
  constructor(apiKey, model = 'gpt-4o-mini') { /* ... */ }

  async explain(prompt, options = {}) {
    // Send prompt to LLM API
    // Handle rate limits with exponential backoff
    // Track token usage for cost monitoring
```

```
    // Return structured response
  }

  countTokens(text) {
    // Estimate token count (roughly: words * 1.3)
  }

  estimateCost(inputTokens, outputTokens) {
    // Calculate API cost
  }
}
```

```
3. Test with a simple prompt:
   "Explain what this function does: function add(a, b) { return a + b; }"
```

Self-Check: - [ ] What is a "token" and why do tokens matter for cost? - [ ] What happens if your prompt exceeds the context window? - [ ] Why would you use `gpt-4o-mini` vs `gpt-4o`?

Day 12: Prompt Engineering for Code Analysis

What to Learn: - System prompts vs user prompts - Few-shot prompting — giving examples for better output - Chain-of-thought prompting — asking the AI to think step by step - Structured output — getting JSON responses from the LLM

Learning Resources:

Resource	Type	Link
Prompt Engineering Guide	Comprehensive	https://www.promptingguide.ai/
OpenAI Prompt Engineering Guide	Official	https://platform.openai.com/docs/guides/prompt-engineering
"Structured Outputs" — OpenAI	Docs	Search "OpenAI structured outputs"

Practice Task:

```
Design and test these prompt templates:

1. FILE_EXPLANATION_PROMPT:
   System: "You are a senior software engineer explaining code to a junior developer..."
   User: "Analyze this file: {filename}\n\nCode:\n{code}\n\nProvide:
          1. Purpose (one sentence)
          2. Key functions and what they do
```

```
        3. Dependencies used and why
        4. Potential improvements"

2. ARCHITECTURE_PROMPT:
   System: "You are a system architect..."
   User: "Given this project structure:\n{fileTree}\n\nAnd these dependencies:\n{depGraph}\n
        Explain:
        1. Overall architecture pattern (MVC, microservices, etc.)
        2. Data flow through the system
        3. Key design decisions"

3. DEPENDENCY_PROMPT:
   "Given these imports: {imports}\n
    Explain what each dependency does and why it's likely used in this context."

Test each prompt with real code from a GitHub repo.
Compare outputs with different temperatures (0.1 vs 0.7 vs 1.0).
```

Self-Check: - [ ] How does temperature affect AI output quality for code explanation? - [ ] What's the difference between zero-shot, few-shot, and chain-of-thought prompting? - [ ] Can you get the AI to return valid JSON consistently?

Day 13: Context Management & Chunking Strategy

What to Learn: - Why you can't send an entire repository to the AI at once - Chunking strategies: by file, by module, by function - Context prioritization — deciding what's most important to send - Summarization chains — summarize parts, then summarize summaries

Learning Resources:

Resource	Type	Link
"Chunking Strategies for LLMs"	Article	Search "LLM chunking strategies"
LangChain Text Splitters	Reference	https://js.langchain.com/docs/modules/data_conn
"Map-Reduce for LLMs"	Pattern	Study how LangChain's map-reduce works

Practice Task:

```
Build a ContextManager module:
```

```
class ContextManager {
  constructor(maxTokens = 8000) { /* ... */ }

  // Strategy 1: Summarize each file individually, then combine
  async analyzeByFile(files, aiEngine) {
    const fileSummaries = [];
    for (const file of files) {
      const summary = await aiEngine.explain(FILE_PROMPT(file));
      fileSummaries.push(summary);
    }
    return await aiEngine.explain(COMBINE_PROMPT(fileSummaries));
  }

  // Strategy 2: Group related files by module/directory
  async analyzeByModule(modules, aiEngine) { /* ... */ }

  // Strategy 3: Send structure first, then deep-dive on key files
  async analyzeHierarchically(structure, aiEngine) {
    const overview = await aiEngine.explain(STRUCTURE_PROMPT(structure));
    const keyFiles = this.identifyKeyFiles(structure);
    const details = await Promise.all(
      keyFiles.map(f => aiEngine.explain(DETAIL_PROMPT(f)))
    );
    return this.combineResults(overview, details);
  }

  identifyKeyFiles(structure) {
    // Entry points, most-imported files, config files
  }
}
```

Self-Check: - [ ] What's the maximum context window for the model you're using? - [ ] How do you decide which files are "important enough" to analyze in detail? - [ ] What's the tradeoff between analyzing more files vs. deeper analysis of fewer files?

Day 14: Putting the AI Pipeline Together

What to Learn: - End-to-end pipeline: clone → traverse → parse → analyze with AI - Output formatting: structured JSON → human-readable markdown - Caching AI responses to avoid redundant API calls - Error handling and fallbacks for AI failures

Practice Task:

`Wire everything together into a complete pipeline:`

```
POST /api/analyze { repoUrl: "https://github.com/expressjs/express" }

Pipeline:
  1. Clone repo → /tmp/repos/express-xxxxx
  2. Traverse files → 47 relevant files found
  3. Parse ASTs → 123 functions, 15 classes extracted
  4. Build dependency graph → entry point: lib/express.js
  5. AI Analysis:
     a. Generate architecture overview
     b. Explain key modules
     c. Identify design patterns
     d. Suggest learning path for contributors
  6. Return formatted result

Expected response:
{
  repository: "expressjs/express",
  overview: "Express.js is a minimal web framework...",
  architecture: { pattern: "Middleware Pipeline", ... },
  keyModules: [...],
  designPatterns: ["Middleware Pattern", "Chain of Responsibility"],
  dependencyGraph: { ... },
  generatedAt: "2026-03-19T..."
}
```

Self-Check: - [ ] Does your pipeline handle a repo with 500+ files without timing out? - [ ] What's the total API cost for analyzing a medium-sized repo? - [ ] Can you analyze the same repo twice without making duplicate API calls?

Phase 4 — Common Mistakes to Avoid

[!WARNING] - **Don't send raw code without context** — always include filename, language, project context - **Don't ignore token limits** — count tokens before sending, truncate if needed - **Don't make unlimited API calls** — set budgets and use caching - **Don't expect perfect output** — always validate and post-process AI responses

Phase 4 — Interview Connection

What interviewers ask: "How would you integrate AI into a production system?" **Your answer now:** You can discuss prompt engineering, context management, token budgets, rate limiting, caching, error handling, and cost optimization.

PHASE 5: Frontend & Visualization

Days 15–20 | Difficulty: (Intermediate)

Learning Objectives

- Build a clean, responsive UI with React and Tailwind CSS
- Connect frontend to backend APIs
- Display code analysis results beautifully
- Visualize dependency graphs

Day 15–16: React Setup & Core UI

What to Learn: - React fundamentals: components, state, props, hooks (`useState`, `useEffect`) - Tailwind CSS for rapid, beautiful styling - Component-based architecture - Form handling and validation

Learning Resources:

Resource	Type	Link
React Official Tutorial	Docs	https://react.dev/learn
Tailwind CSS Docs	Docs	https://tailwindcss.com/docs
"React in 100 Seconds" — Fireship	Video (2m)	Search "Fireship React 100 seconds"
"Full React Course" — freeCodeCamp	Video (12hr)	Search "freeCodeCamp React 2024"
"Tailwind CSS Full Course" — Dave Gray	Video	Search "Dave Gray Tailwind" on YouTube

Practice Task:

```
Build these components:

1. <RepoInput />
   - URL input field with validation
   - "Analyze" button with loading state
   - Error display for invalid URLs

2. <AnalysisStatus />
```

```
   - Progress bar / spinner during analysis
   - Step-by-step status updates:
      Repository cloned
      Files discovered (47 files)
      Analyzing code structure...
      Generating AI explanation

3. <Layout />
   - Dark mode header with project branding
   - Clean, centered content area
   - Footer with GitHub link
```

Day 17–18: Results Display & Markdown Rendering

What to Learn: - Rendering markdown content in React - Syntax highlighting for code blocks - Responsive card/section layouts - Tab-based navigation between analysis sections

Learning Resources:

Resource	Type	Link
react-markdown	Library	https://www.npmjs.com/package/react-markdown
react-syntax-highlighter	Library	https://www.npmjs.com/package/react-syntax-highlighter
Prism.js themes	Styling	https://prismjs.com/

Practice Task:

```
Build the results display:

1. <AnalysisResult />
   - Tabbed interface: Overview | Modules | Dependencies | AI Insights

2. <Overview Tab>
   - Repository name, language, stats
   - Architecture pattern identification
   - High-level description (rendered as markdown)

3. <Modules Tab>
   - Collapsible file tree
   - Click a file → see its functions, classes, imports
   - AI-generated explanation for each module
```

```
4. <Dependencies Tab>
   - Visual dependency graph (use vis.js, d3.js, or mermaid)
   - Highlight entry points and leaf nodes
   - Click a node → see module details

5. <AI Insights Tab>
   - Design patterns detected
   - Code quality observations
   - Learning path suggestions
```

Day 19–20: API Integration & State Management

What to Learn: - Fetching data from your backend API - Loading, error, and success states - Polling or WebSockets for long-running analysis - Local state management with React hooks

Learning Resources:

Resource	Type	Link
Fetch API — MDN	Docs	https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
Axios npm	Library	https://www.npmjs.com/package/axios
React Query / TanStack Query	Advanced	https://tanstack.com/query
"React Hooks Explained" — Web Dev Simplified	Video	Search "Web Dev Simplified React hooks"

Practice Task:

```
Connect everything:

1. useAnalysis() custom hook:
   - Handle submit → POST /api/analyze
   - Poll GET /api/analyze/:id every 2 seconds for status
   - Update UI with progress
   - Display results when complete

2. Error Boundaries:
   - Network errors → "Connection failed, retrying..."
   - Invalid repo → "Repository not found"
   - AI failure → "Analysis partially complete" (show what you have)

3. URL-based routing:
   - / → Landing page with input
```

```
   - /analysis/:id → Results page
   - Share analysis via URL
```

Self-Check: - [ ] Does the UI update in real-time during analysis? - [ ] What happens if the user closes the tab and returns later? - [ ] Is the UI fully responsive (mobile + desktop)?

Phase 5 — Common Mistakes to Avoid

> [!WARNING] - **Don't build the UI first** — the backend should be working before you touch frontend - **Don't ignore loading states** — analysis takes time, keep the user informed - **Don't forget error handling** — show user-friendly messages, not raw API errors - **Don't skip mobile responsiveness** — recruiters check on their phones

Phase 5 — Interview Connection

> **What interviewers ask:** "Walk me through a feature from frontend to backend." **Your answer now:** You can trace the complete flow: user submits URL → API call → repo clone → AST parsing → AI analysis → results rendered in React.

PHASE 6: Final Polish & Production Readiness

Days 21–25 | **Difficulty: (Intermediate)**

Learning Objectives

- Add proper error handling at every layer
- Optimize performance and user experience
- Write documentation that impresses recruiters
- Deploy the application

Day 21–22: Error Handling & Edge Cases

Tasks:

```
1. Backend Error Handling:
   - Invalid GitHub URLs → 400 with clear message
   - Private repositories → 403 "Repository is private or doesn't exist"
   - Clone timeouts → 408 "Repository too large or network timeout"
   - Parse failures → partial results with warnings
```

```
   - AI API failures → cached/fallback responses

2. Frontend Error Handling:
   - Network failures → retry with backoff
   - Empty results → helpful "No analyzable code found" message
   - Long analysis → "Still working... Large repos take 2-3 minutes"
```

Day 23: Performance Optimization

Tasks:

```
1. Backend:
   - Implement caching (analyzed repos stored for 24hrs)
   - Parallelize independent operations (file reading, AI calls)
   - Add request queuing for AI API calls
   - Stream results to frontend as they become available

2. Frontend:
   - Lazy load heavy components (dependency graph)
   - Debounce URL input validation
   - Add skeleton loading states
   - Optimize bundle size
```

Day 24: Documentation & README

Tasks:

```
Create a stellar README.md:

1. Project banner/logo
2. One-line description
3. Screenshots/GIFs of the tool in action
4. Features list
5. Tech stack with brief explanations of WHY each was chosen
6. Architecture diagram
7. Setup instructions (step by step)
8. API documentation
9. Contributing guidelines
10. Future roadmap
```

Day 25: Deployment & Demo Prep

Deployment Resources:

Resource	Type	Link
Vercel (Frontend)	Free Hosting	https://vercel.com/
Railway (Backend)	Free Hosting	https://railway.app/
Render (Alternative)	Free Hosting	https://render.com/
Docker Basics	Container	https://docs.docker.com/get-started/

Tasks:

```
1. Deploy backend to Railway/Render
2. Deploy frontend to Vercel
3. Set up environment variables in production
4. Test the full flow with 3 different repos
5. Record a 2-minute demo video
6. Create a LinkedIn post about what you built and learned
```

Detailed Day-by-Day Schedule

Day	Phase	Focus	Deliverable
1	Phase 1	Node.js core, event loop	Basic HTTP server
2	Phase 1	Express.js, REST APIs	3 API endpoints
3	Phase 1	Project setup, Git	Clean repo with scripts
4	Phase 2	Git clone via Node, fs basics	RepoCloner module
5	Phase 2	Recursive traversal	FileTraverser module
6	Phase 2	File filtering, content reading	FileFilter + ContentReader
7	Phase 3	AST fundamentals	AST Explorer exercises
8	Phase 3	Building the parser	CodeParser module
9	Phase 3	Dependency graphs	DependencyAnalyzer module
10	Phase 3	Multi-language, summary	Unified CodeStructure output
11	Phase 4	LLM basics, API setup	AIEngine module

Day	Phase	Focus	Deliverable
12	Phase 4	Prompt engineering	3 prompt templates tested
13	Phase 4	Context management	ContextManager module
14	Phase 4	Full AI pipeline	End-to-end pipeline working
15	Phase 5	React + Tailwind setup	RepoInput component
16	Phase 5	Core UI components	AnalysisStatus component
17	Phase 5	Results display	AnalysisResult with tabs
18	Phase 5	Markdown rendering	Code highlighting working
19	Phase 5	API integration	Frontend connected to backend
20	Phase 5	State management	Polling + real-time updates
21	Phase 6	Backend error handling	All edge cases handled
22	Phase 6	Frontend error handling	User-friendly error UX
23	Phase 6	Performance optimization	Caching + parallelization
24	Phase 6	Documentation	README + API docs
25	Phase 6	Deployment	Live demo + video

Master Resource Library

Core Technologies

Technology	Best Resource	Link
Node.js	Official Docs	https://nodejs.org/en/docs
Express.js	Official Guide	https://expressjs.com/
React	Official Tutorial	https://react.dev/learn
Tailwind CSS	Official Docs	https://tailwindcss.com/docs

Code Analysis

Technology	Best Resource	Link
AST Explorer	Interactive Tool	https://astexplorer.net/
Babel Parser	npm Docs	https://babeljs.io/docs/babel-parser
Tree-sitter	Official Docs	https://tree-sitter.github.io/tree-sitter/

AI / LLM

Technology	Best Resource	Link
OpenAI API	Official Docs	https://platform.openai.com/docs
Google Gemini	Official Docs	https://ai.google.dev/docs
Prompt Engineering	DAIR.AI Guide	https://www.promptingguide.ai/

Recommended YouTube Channels

Channel	Best For
Fireship	Quick tech overviews (100-second series)
Traversy Media	Full project tutorials
The Net Ninja	Step-by-step Node & React courses
Dave Gray	Comprehensive Express & React courses
Web Dev Simplified	React hooks and patterns explained simply
ByteByteGo	System design concepts (visual)
Programming with Mosh	Node.js deep dives
freeCodeCamp	Long-form complete courses

Recommended Books (Optional)

Book	Author	Why
"Node.js Design Patterns"	Mario Casciaro	Deep Node.js understanding
"You Don't Know JS"	Kyle Simpson	JavaScript mastery (free online)
"Designing Data-Intensive Applications"	Martin Kleppmann	System design thinking

Interview Preparation Cheat Sheet

After completing this project, you can confidently answer:

Interview Topic	Your Experience From This Project
REST API Design	Built 5+ endpoints with proper status codes and error handling
Async Programming	Used async/await, Promises, event loop knowledge throughout
File System Operations	Recursive traversal, streaming, filtering 1000s of files
Data Structures	Dependency graphs, trees (AST, file tree), maps
System Design	Designed a pipeline: clone → parse → analyze → present
AI/ML Integration	Prompt engineering, token management, context windowing
Caching Strategies	Implemented caching for AI responses and repo analysis
Error Handling	Graceful degradation, retry strategies, user-friendly errors
Frontend-Backend	Full-stack: React UI Express API AI Service
Cost Optimization	Token budgeting, result caching, smart chunking

---

Bonus: Stretch Goals (After Day 25)

If you finish early or want to go further:

- ☐ **Add authentication** — GitHub OAuth to analyze private repos
- ☐ **Add a database** — Store analysis history (PostgreSQL or MongoDB)
- ☐ **Add RAG (Retrieval-Augmented Generation)** — Use vector embeddings (ChromaDB / Pinecone) for semantic code search
- ☐ **Add multi-language support** — Python, Java, Go, Rust via Tree-sitter
- ☐ **Add comparison mode** — Compare two repos or two branches
- ☐ **Add code review mode** — AI suggests improvements for analyzed code
- ☐ **Publish as a GitHub Action** — Auto-analyze repos on push
- ☐ **Add WebSocket** — Real-time analysis progress instead of polling

---

> [!TIP] **Learning Tip:** Don't just code — write about what you learned each day. A blog post series "Building an AI Code Analyzer in 25 Days" can be as valuable as the project itself for your career.
>
> [!IMPORTANT] **Timeline is flexible!** If a phase takes longer, that's completely fine. Understanding deeply is more valuable than

finishing fast. Some days you might breeze through, others might take 2-3 days. Adjust to YOUR pace.

Created on March 19, 2026 — Good luck building!